AUTOMATED PODCAST PROJECT DOCUMENTATION

OVERVIEW:

The rapid growth of digital media has resulted in a massive increase in audio content such as podcasts, audiobooks, online lectures, interviews, webinars, and recorded meetings. While audio content is rich in information, it is difficult to analyze, search, and summarize without converting it into text. Listening to long recordings to find specific information is time-consuming and inefficient. This creates a strong need for automated systems that can convert speech into structured, searchable, and analyzable text data.

The Automated Podcast Transcription and Topic Segmentation Dashboard is designed to address this challenge by providing a complete end-to-end solution that transforms raw audio into meaningful textual insights. The system automatically cleans audio, generates transcripts, segments the text into logical topics, extracts important keywords, and visualizes the results using an interactive dashboard. The project integrates concepts from digital signal processing, natural language processing (NLP), data visualization, and web application development.

The primary goal of this project is to reduce human effort in manually transcribing and analyzing audio content. Traditional transcription requires trained personnel, consumes significant time, and introduces human error. In contrast, this system automates the workflow, allowing users to process audio content efficiently and consistently. Once audio is converted into text, the content becomes searchable, analyzable, and reusable for reporting, indexing, and knowledge discovery.

At the heart of this project lies the transformation pipeline that processes audio data into structured information. Raw audio files are first cleaned to reduce noise and improve clarity. This step improves the accuracy of downstream speech recognition. The cleaned audio is then converted into text using speech-to-text mechanisms. The transcript undergoes preprocessing such as removing unwanted symbols, formatting inconsistencies, headers, and irrelevant text. This cleaned text is further segmented into meaningful blocks or topics, making it easier for users to navigate long documents.

Keyword extraction plays a critical role in summarizing large volumes of text. By identifying frequently occurring and meaningful words, the system highlights the core concepts discussed in the audio. These keywords enable quick content understanding, indexing, and search functionality. Users can easily locate specific topics without scanning the entire transcript manually.

Visualization enhances the interpretability of the extracted data. The system generates bar charts showing word frequency distributions and word clouds that visually represent the dominance of key terms. These visual representations allow users to quickly grasp trends and patterns in the content. For

example, a podcast episode discussing technology trends may show words such as “AI,” “data,” “cloud,” and “automation” prominently in the word cloud, enabling instant insight into the subject matter.

A major feature of this project is the interactive dashboard built using Streamlit. The dashboard acts as the central interface where users can interact with all components of the system. Through intuitive tabs, users can upload or select transcript files, view transcription results, examine segmented text, analyze keywords, visualize charts, and play cleaned audio files. This graphical interface eliminates the need for users to interact directly with command-line tools or scripts, making the system accessible even to non-technical users.

From an academic perspective, this project demonstrates practical implementation of AI-based text processing pipelines. It combines real-world datasets, open-source libraries, and modular architecture to create a scalable and maintainable system. Each processing stage is independent yet integrated, enabling easy debugging, enhancement, and reuse. This modular design is beneficial for students and researchers who want to extend the system with additional features such as sentiment analysis, speaker identification, or automatic summarization.

The system also supports reproducibility and experimentation. By organizing data into structured folders such as audio_raw, audio_processed, transcripts, segmented, and keywords, the project ensures traceability of outputs and easy validation of intermediate results. Developers can inspect each stage of the pipeline independently to analyze performance and accuracy.

This transparency is essential for testing, debugging, and future enhancements.

Another important aspect of this project is its relevance to accessibility and information management. Audio content is often inaccessible to hearing-impaired individuals or those who prefer reading. By converting speech into text, the system makes audio content more inclusive and usable. Furthermore, searchable transcripts allow users to retrieve specific information quickly, increasing productivity and reducing information overload.

The dashboard-oriented approach improves usability and adoption. Instead of providing raw text files or command-line outputs, the system presents results in a visually structured and user-friendly manner. This aligns with modern software development practices where usability and accessibility are critical success factors. The interface supports rapid exploration of content, making it suitable for demonstrations, academic evaluations, and practical usage scenarios.

Scalability is another consideration addressed by this project. Although the current implementation is designed for local execution, the modular architecture allows future deployment on cloud platforms. With additional infrastructure, the system can be extended to handle large-scale datasets, real-time audio streams, and collaborative usage. The current version serves as a proof-of-concept demonstrating feasibility and functionality.

Security and data privacy considerations can also be incorporated into future versions. Audio and transcripts may contain sensitive information in enterprise environments. Controlled access, encryption, and secure storage mechanisms

can be added as enhancements. While not implemented in the current version, the architecture allows such integrations.

From a learning perspective, this project offers hands-on exposure to several important concepts: audio preprocessing, text normalization, frequency analysis, visualization techniques, file system management, and web application deployment. It bridges the gap between theoretical knowledge and practical implementation. Students gain experience in building real-world pipelines that combine multiple domains.

In summary, the Automated Podcast Transcription and Topic Segmentation Dashboard provides a comprehensive solution for converting unstructured audio data into structured and actionable insights. It simplifies audio analysis through automation, improves accessibility through text conversion, enhances understanding through segmentation and visualization, and delivers usability through a modern web-based interface. The project demonstrates the power of integrating AI technologies with software engineering principles to solve real-world problems efficiently and effectively.

APPROACH

The development of the Automated Podcast Transcription and Topic Segmentation Dashboard follows a systematic and modular approach that converts unstructured audio data into structured and analyzable information. The project pipeline is

designed to ensure clarity, scalability, maintainability, and ease of debugging. Each stage of the system focuses on a specific responsibility, while seamlessly integrating with subsequent stages to form an end-to-end workflow.

The approach begins with data acquisition, followed by audio preprocessing, transcription, text preprocessing, segmentation, keyword extraction, visualization, and user interface integration. This layered architecture ensures that improvements or changes in one module do not significantly affect the overall system. The following subsections describe each stage of the approach in detail.

1. Data Acquisition and Organization

The first step in the approach involves obtaining audio data and organizing it in a structured directory format. Audio files are placed in the audio_raw folder to preserve original data integrity. Keeping raw data unchanged is a best practice in data engineering because it allows reprocessing without losing original information.

The project supports common audio formats such as WAV and MP3. Each audio file is named consistently so that it can be matched with its corresponding transcript, segmented text, and keyword files later in the pipeline. Proper file naming conventions ensure traceability across processing stages and reduce the risk of mismatched data.

The folder-based architecture provides logical separation of concerns:

Raw audio is isolated from processed outputs.

Transcripts are stored separately from segmentation results.

Keywords and visual outputs remain organized for analysis.

This organization simplifies automation, debugging, and scalability.

2. Audio Preprocessing and Cleaning

Audio quality significantly impacts transcription accuracy. Background noise, inconsistent volume levels, and distortions reduce speech recognition performance. Therefore, the second stage of the approach focuses on audio preprocessing.

Audio cleaning involves:

Normalizing audio volume levels.

Removing silence or unnecessary noise.

Converting audio into a consistent sampling rate and format.

The cleaned audio files are stored in the audio_processed folder. This ensures the raw audio remains untouched while processed files are optimized for speech recognition.

Audio preprocessing improves transcription reliability and ensures consistent downstream processing. The separation between raw and processed audio enables performance comparison and iterative enhancement.

3. Speech-to-Text Transcription

Once the audio is cleaned, it is passed through a speech-to-text process. This step converts spoken words into textual representation. The generated transcript forms the foundation for all subsequent analysis tasks.

The transcription output is saved as a text file inside the transcripts folder. Each transcript is named consistently with the audio file name to ensure traceability.

Transcription accuracy depends on:

Audio clarity.

Speaker accent and pronunciation.

Background noise.

Audio length.

While automated transcription may not achieve perfect accuracy, it provides sufficient baseline quality for segmentation and keyword extraction.

4. Text Preprocessing

Raw transcripts often contain irregular formatting, punctuation noise, repeated symbols, filler words, and inconsistent spacing. The preprocessing stage standardizes text to improve segmentation and keyword extraction accuracy.

Key preprocessing steps include:

Converting text to lowercase.

Removing special characters and digits.

Normalizing whitespace.

Removing unnecessary headers or metadata.

Cleaning formatting artifacts.

Text preprocessing reduces noise and improves consistency across the dataset.

5. Topic Segmentation

Segmentation divides long transcripts into smaller meaningful blocks. This improves readability and helps users navigate lengthy audio content.

The segmentation approach in this project uses sentence-based grouping. Sentences are grouped into paragraphs based on logical intervals or sentence count. Each segment represents a coherent block of discussion.

Segmented text is saved into the segmented folder. This enables direct access from the dashboard.

Although the current segmentation approach is rule-based, it establishes a foundation for future intelligent segmentation using machine learning or semantic similarity.

6. Keyword Extraction

Keyword extraction identifies important terms that summarize the content. Frequency-based analysis is used to count word occurrences after removing stopwords and irrelevant symbols.

The extracted keywords represent dominant topics in the transcript. Keywords enable:

Quick understanding of content.

Search and indexing.

Visualization.

Keyword files are stored in the keywords folder for direct access by the UI.

7. Visualization

Visualization transforms textual data into graphical insights. The project generates:

Bar charts showing word frequency.

Word clouds highlighting dominant keywords.

Visualization improves interpretability and supports data-driven decision making.

8. User Interface Integration

The Streamlit dashboard integrates all modules into a single interactive platform. Users can:

Select transcripts.

View segmented text.

View keywords.

Play cleaned audio.

Analyze visualizations.

The UI abstracts technical complexity and improves accessibility.

9. Error Handling and Validation

Each module includes validation checks to ensure file existence and handle missing data gracefully. Logging and user-friendly error messages improve reliability.

10. Modularity and Scalability

Each processing stage is modular, enabling independent enhancement and scalability.

11. Testing Strategy

System testing validates end-to-end functionality and integration stability.

12. Optimization and Performance

Optimized file handling and modular processing minimize resource usage.

13. Security and Data Integrity

Raw data preservation ensures traceability and reproducibility.

Conclusion

The approach demonstrates a practical and scalable architecture for audio-to-text analytics and dashboard-driven visualization.

TECH STACK

The success of the Automated Podcast Transcription and Topic Segmentation Dashboard depends heavily on selecting the right technologies that balance performance, ease of development, scalability, and maintainability. This project integrates multiple layers of technology including programming language, audio processing tools, natural language processing libraries, data visualization frameworks, user interface tools, and development utilities. Each technology has been chosen based on reliability, community support, compatibility, and ease of integration.

The following sections describe each component of the technology stack in detail and explain why it was selected and how it contributes to the overall system.

1. Programming Language – Python

Python serves as the core programming language for this project. It is widely used in data science, artificial intelligence, machine learning, and web development. Python provides a rich ecosystem of libraries for audio processing, text processing, visualization, and web application development, making it an ideal choice for this project.

Advantages of Python:

Ease of Learning: Python has simple syntax, making it easy to write, read, and maintain code.

Extensive Libraries: Python offers thousands of open-source libraries that simplify development.

Cross-Platform: Python applications run on Windows, Linux, and macOS.

Rapid Development: Faster prototyping compared to compiled languages.

Community Support: Large developer community ensures quick troubleshooting and continuous improvements.

In this project, Python manages file operations, executes processing scripts, performs text analysis, generates visualizations, and runs the Streamlit dashboard.

2. Audio Processing Tools

Audio preprocessing plays a critical role in improving transcription accuracy. Several audio-related libraries and tools are used.

FFmpeg

FFmpeg is a powerful open-source multimedia framework used for audio conversion, normalization, and noise handling. It supports a wide range of audio formats and codecs.

Usage in project:

Convert audio into standardized formats.

Normalize audio volume.

Remove unwanted metadata.

Benefits:

High performance

Supports many formats

Reliable and fast processing

Pydub

Pydub is a Python library for audio manipulation.

Usage:

Load audio files.

Modify audio properties.

Export processed audio.

Benefits:

Easy Python integration

Simple API

Supports WAV, MP3, FLAC

3. Natural Language Processing (NLP) Libraries

Text processing is a core component of this project. NLP libraries enable tokenization, cleaning, segmentation, and keyword extraction.

NLTK (Natural Language Toolkit)

NLTK provides comprehensive NLP utilities.

Usage:

Tokenization

Stopword removal

Text preprocessing

Benefits:

Academic-grade NLP toolkit

Extensive documentation

Widely used in research

Regular Expressions (Regex)

Regex is used for pattern matching and text cleaning.

Usage:

Remove punctuation and numbers

Normalize spaces

Clean noisy text

4. Data Processing and Management

OS Module

Python’s OS module is used for directory navigation, file handling, and automation.

Usage:

Create folders

Read and write files

Manage paths

Collections Module

The Counter class is used for frequency analysis.

Usage:

Count word occurrences

Generate keyword frequency lists

5. Visualization Tools

Visualization makes data easier to interpret.

Matplotlib

Matplotlib is a plotting library.

Usage:

Bar charts for keyword frequency

Custom plots

Benefits:

High customization

Stable library

Widely adopted

WordCloud

WordCloud visually represents frequent words.

Usage:

Generate word cloud images

Highlight dominant keywords

Benefits:

Intuitive visualization

Easy integration

6. User Interface – Streamlit

Streamlit is a Python framework for building interactive web applications.

Usage:

Dashboard interface

Tabs and layouts

File selection

Audio playback

Graph display

Benefits:

Rapid UI development

No front-end coding required

Easy deployment

7. Development Environment

Visual Studio Code (VS Code)

VS Code is used for coding and debugging.

Features:

Syntax highlighting

Debugging tools

Git integration

Command Line Interface

Used to execute scripts and manage environments.

8. Version Control – Git and GitHub

Git manages version tracking.

Usage:

Track changes

Collaboration

Backup code

GitHub hosts the repository.

9. Operating System

Windows OS is used for development and testing.

10. File Formats

Audio: WAV, MP3

Text: TXT

Images: PNG (visualizations)

Scripts: PY

11. Security and Dependency Management

Virtual environments recommended.

Library version control.

12. Performance Considerations

Local processing optimized.

Lightweight libraries.

13. Scalability Potential

Can migrate to cloud.

Can integrate ML models.

14. Compatibility

Cross-platform Python compatibility.

15. Maintainability

Modular architecture.

Conclusion

The chosen tech stack ensures reliability, scalability, ease of development, and real-world applicability. Python acts as the backbone while Streamlit provides a modern interface, and NLP libraries deliver analytical power. Together they form a robust platform for audio analytics.

SETUP

Setting up the Automated Podcast Transcription and Topic Segmentation Dashboard requires installing the required software, configuring the development environment, organizing the project directory structure, installing dependencies, and validating the setup through test runs. A well-documented setup process ensures reproducibility, smooth onboarding for new users, and stable system execution.

This section provides a step-by-step guide to install, configure, and verify the system on a local machine.

1. System Requirements

Before installation, ensure that the system meets the minimum requirements:

Operating System: Windows 10 or later (Linux/macOS supported with minor changes)

Processor: Minimum Intel i3 or equivalent

RAM: At least 8 GB recommended

Storage: Minimum 5 GB free space

Internet Connection: Required for library installation

2. Installing Python

Python is the primary programming language for this project.

Step-by-step installation:

Visit:

Copy code

https://www.python.org

Download Python 3.8 or higher.

During installation, enable:

Copy code

✓ Add Python to PATH

Complete the installation.

Verify installation:

Copy code

```
python --version
```

3. Installing Package Manager (pip)

pip comes bundled with Python. Verify installation:

Copy code

```
python -m pip --version
```

Upgrade pip:

Copy code

```
python -m pip install --upgrade pip
```

4. Creating Project Directory Structure

Create the following folder structure:

Copy code

```
project/
│
├── audio_raw/
├── audio_processed/
├── transcripts/
├── segmented/
```

├── keywords/

├── script/

└── ui/

This structure maintains modular organization and clean data flow.

5. Installing Required Libraries

Navigate to the project folder:

Copy code

cd project

Install dependencies:

Copy code

python -m pip install streamlit matplotlib wordcloud nltk pydub librosa pandas

6. Downloading NLP Resources

Open Python interpreter and run:

Copy code

Python

import nltk

nltk.download("punkt")

nltk.download("stopwords")

7. Installing FFmpeg

Audio preprocessing requires FFmpeg.

Installation steps (Windows):

Download FFmpeg from official site.

Extract files.

Add FFmpeg bin folder to system PATH.

Verify installation:

Copy code

```
ffmpeg -version
```

8. Adding Audio Files

Place audio files into:

Copy code

```
audio_raw/
```

Supported formats: WAV, MP3

9. Running Audio Cleaning

Navigate to script folder:

Copy code

```
cd script
python audio_clean.py
```

Verify cleaned audio in:

Copy code

audio_processed/

10. Running Transcription

Run transcription script:

Copy code

python transcript.py

Check:

Copy code

transcripts/

11. Running Segmentation

Copy code

python segment_text.py

Verify:

Copy code

segmented/

12. Running Keyword Extraction

Copy code

python extract_keywords.py

Verify:

Copy code

```
keywords/
```

13. Launching Dashboard

Navigate to UI folder:

Copy code

```
cd ui
streamlit run app.py
```

Access in browser:

Copy code

```
http://localhost:8501
```

14. Validation Checklist

Audio appears in Cleaned Audio tab

Transcript visible

Segmented text visible

Keywords visible

Visualizations render

15. Common Setup Mistakes

Missing PATH configuration

Incorrect folder names

Missing dependencies

16. Backup and Version Control

Use GitHub for backup and versioning.

17. Environment Best Practices

Use virtual environment.

Conclusion

Following this setup ensures stable execution and reproducibility.

USAGE

The Usage section explains how end users and developers interact with the Automated Podcast Transcription and Topic Segmentation Dashboard after successful installation and setup. It describes the complete operational flow starting from preparing input data to visualizing analytical results in the dashboard. This section ensures that users can confidently operate the system without requiring deep technical expertise.

The system supports both command-line execution for preprocessing tasks and a graphical user interface for interactive analysis. Users can choose their preferred workflow depending on their technical background and operational requirements.

1. Preparing Input Audio Files

The first step in using the system is preparing the audio files. Users place raw podcast audio files inside the audio_raw folder. Supported formats include WAV and MP3. Audio files should ideally be clear and free from excessive background noise to improve transcription accuracy.

Users should follow consistent naming conventions for audio files because the system uses file name matching to link audio, transcripts, segmented files, and keyword files.

2. Audio Cleaning Execution

After placing audio files in audio_raw, users run the audio cleaning script located in the script folder.

Copy code

```
python audio_clean.py
```

This script processes each audio file and generates cleaned audio outputs in the audio_processed folder. Users should verify the existence of processed audio files and confirm that audio playback works correctly.

3. Transcription Generation

The transcription script converts cleaned audio into text.

Copy code

```
python transcript.py
```

The generated transcripts are saved in the transcripts folder. Users can open these text files manually to verify correctness before proceeding further.

4. Text Preprocessing

If preprocessing scripts exist, users run them to remove noise and normalize text formatting. This step ensures segmentation and keyword extraction produce meaningful results.

5. Topic Segmentation

Segmentation splits long transcripts into smaller blocks.

Copy code

python segment_text.py

Segmented files appear in the segmented folder. Each file contains grouped sentences representing coherent discussion blocks.

6. Keyword Extraction

Users extract keywords by running:

Copy code

python extract_keywords.py

Keyword files appear in the keywords folder.

7. Launching the Dashboard

Navigate to UI folder:

Copy code

cd ui

streamlit run app.py

8. Using the Dashboard Interface

Upload Tab

Users can upload new transcript files which are automatically stored.

Transcript Tab

Displays full transcript text.

Topic Segments Tab

Displays segmented text.

Keywords Tab

Displays extracted keywords.

Visualization Tab

Displays bar chart and word cloud.

Cleaned Audio Tab

Plays processed audio.

9. Searching Keywords

Users can manually search keywords inside transcript.

10. Interpreting Visualizations

Charts help understand frequency trends.

11. Multiple File Handling

Users can analyze multiple transcripts.

12. Updating Data

Users can rerun scripts when new data is added.

13. Error Handling

UI displays warnings for missing files.

14. Performance Tips

Use moderate audio size.

15. Best Practices

Keep folder clean.

Conclusion

Usage instructions ensure smooth operation and usability.

TROUBLESHOOTING

The troubleshooting section provides guidance for identifying, diagnosing, and resolving common issues encountered while installing, configuring, and running the Automated Podcast Transcription and Topic Segmentation Dashboard. Because the project integrates multiple software components such as audio processing tools, NLP libraries, visualization frameworks, and a web-based UI, users may encounter various runtime errors or configuration problems. This section ensures that users can quickly resolve problems and maintain system stability.

1. Python Installation Issues

Problem

Python command is not recognized in the terminal.

Cause

Python is not added to the system PATH.

Solution

Reinstall Python and enable the option:

Copy code

✓ Add Python to PATH

Verify using:

Copy code

python --version

2. pip Not Recognized

Problem

pip command not found.

Solution

Use:

Copy code

python -m pip install <package>

3. Missing Library Errors

Problem

ModuleNotFoundError appears.

Solution

Install missing module using pip.

4. Streamlit Not Launching

Problem

Browser does not open.

Solution

Restart terminal or check firewall.

5. Audio Not Playing

Problem

Audio player shows error.

Cause

Incorrect audio path or unsupported format.

Solution

Ensure audio exists in audio_processed.

6. FFmpeg Not Detected

Problem

FFmpeg command not recognized.

Solution

Add FFmpeg bin folder to PATH.

7. Segmented File Missing

Problem

Segmented text not displayed.

Solution

Run segment_text.py again.

8. Keyword File Missing

Problem

Keywords not visible.

Solution

Run extract_keywords.py.

9. Visualization Errors

Problem

Charts not rendering.

Solution

Install matplotlib and wordcloud.

10. Permission Errors

Problem

Permission denied.

Solution

Close files or run terminal as admin.

11. Encoding Errors

Problem

UnicodeDecodeError.

Solution

Open file using UTF-8 encoding.

12. File Path Errors

Problem

FileNotFoundError.

Solution

Verify folder structure.

13. Audio Processing Errors

Problem

Conversion fails.

Solution

Verify FFmpeg installation.

14. Memory Issues

Problem

Application freezes.

Solution

Reduce audio size.

15. Browser Cache Issues

Problem

UI not refreshing.

Solution

Clear cache or restart Streamlit.

16. Virtual Environment Issues

Problem

Wrong Python environment.

Solution

Activate correct environment.

17. GitHub Upload Issues

Problem

Large files not uploading.

Solution

Use .gitignore.

18. Debugging Tips

Use print statements and logs.

19. Logging Best Practices

Log errors and warnings.

20. When to Seek Help

Consult documentation.

Conclusion

Troubleshooting ensures smooth operation and user confidence.

LIMITATIONS

While the Automated Podcast Transcription and Topic Segmentation Dashboard demonstrates a functional and scalable pipeline for transforming audio into structured textual insights, it is important to recognize that the system has certain limitations. These limitations arise from technological

constraints, resource availability, algorithmic simplicity, dependency reliance, and deployment scope. Understanding these limitations helps stakeholders set realistic expectations and identify opportunities for future improvements.

1. Transcription Accuracy Constraints

Automated speech recognition systems are inherently imperfect. The transcription accuracy depends on several factors including audio clarity, speaker accent, speech speed, background noise, microphone quality, and pronunciation. In noisy environments or when multiple speakers overlap, transcription errors increase. Technical jargon, domain-specific vocabulary, or accented speech may not be accurately recognized by generic speech-to-text engines.

Because the system uses automated transcription without manual correction, minor inaccuracies propagate into segmentation and keyword extraction stages. Incorrect words can affect frequency counts and segmentation quality. This limits the precision of downstream analytics.

2. Limited Audio Noise Handling

Although audio preprocessing improves clarity, advanced noise reduction and echo cancellation are not fully implemented. The current approach performs basic normalization and conversion. Complex acoustic environments such as outdoor recordings, echo-heavy rooms, or background music remain challenging.

Professional-grade audio enhancement requires advanced signal processing or machine learning-based denoising, which is beyond the scope of the current implementation.

3. Rule-Based Segmentation

Segmentation is implemented using rule-based grouping of sentences rather than semantic understanding. This may not always reflect true topic boundaries. Conversations often shift topics gradually, which rule-based segmentation may fail to capture accurately.

Advanced topic modeling or embedding-based segmentation techniques could improve accuracy but are not implemented in the current version.

4. Basic Keyword Extraction

Keyword extraction relies primarily on word frequency analysis. This approach does not capture semantic importance, context, or phrase-level meaning. High-frequency words may not always represent meaningful concepts. Synonyms and multi-word phrases are not handled optimally.

More advanced NLP methods such as TF-IDF, Named Entity Recognition, or transformer-based models could provide richer insights.

5. Limited Language Support

The system is primarily optimized for English language content. Multilingual audio support is not implemented. Speech recognition accuracy and NLP processing may degrade significantly for other languages.

Supporting multiple languages would require language detection, multilingual NLP pipelines, and localized models.

6. Scalability Constraints

The system runs locally on a single machine. Large datasets or long audio files may consume significant processing time and

memory. Parallel processing and distributed computing are not implemented.

Real-time transcription is also not supported.

7. UI Customization Limitations

The Streamlit UI offers limited customization compared to full-scale web frameworks. Complex layouts, animations, and advanced interactivity are constrained by framework capabilities.

8. Lack of Speaker Identification

The system does not perform speaker diarization. Conversations involving multiple speakers are treated as a single continuous transcript. This limits speaker-based analytics.

9. Absence of Sentiment Analysis

Sentiment analysis is not included, limiting emotional insights.

10. Manual Workflow Dependency

Users must manually execute scripts in sequence.

11. Dataset Dependency

Results depend heavily on dataset quality.

12. Security and Privacy

Data encryption and authentication are not implemented.

13. Evaluation Metrics Limited

Only basic evaluation is performed.

14. Maintenance Overhead

Library updates may break compatibility.

15. Internet Dependency for Installation

Initial setup requires internet access.

16. Platform Dependency

Primarily tested on Windows.

17. Visualization Simplicity

Limited advanced analytics.

18. Limited Automation

No scheduling or automation.

19. Error Recovery

Limited automatic recovery.

20. User Training Required

Basic technical knowledge needed.

Conclusion

Understanding these limitations enables informed usage and provides direction for future enhancements.
